

AGV 智能调度算法全景 与可复用母模块 V2

全面算法地图 · 固定母模型 · 固定应用场景 · 混合架构 · 可运行 Python
代码

本书解决的问题

不是背算法名字，而是建立一套可反复套用的判断框架：问题在哪一层？属于哪种求解范式？
固定母循环是什么？哪些模块能复用？怎样和其他算法混合？

版本：2026-07-19 适用阶段：AGV 调度入门 → 项目化研究

使用说明

这份手册按“先母框架、再算法家族、最后混合与代码”的顺序编排。第一次只需看第 1、2、5、7 章；第二次再按你的 AGV 问题回查算法家族；真正写项目时打开配套 .py 文件替换模块。

学习层次	你要达到的程度	不要陷入的误区
第 1 层：能选	说清决策层、约束、动作空间和评价指标	只看论文标题里的算法缩写
第 2 层：能换	保持统一接口，替换规则/MILP/元启发式/PPO/MARL	每换算法就重写环境和评估
第 3 层：能混	理解谁负责派工、谁负责路径、谁负责冲突、谁负责学习	把所有组件都称为‘调度算法’
第 4 层：能证	多随机种子、强基线、消融、约束违例和计算时间	只展示一条 reward 曲线

配套文件

完整母模块	agv_algorithm_mother_modules_v2.py：标准库即可运行，含规则、路径、冲突、经验、Q-learning、混合协调和评估。
自动测试	test_agv_algorithm_mother_modules_v2.py：验证候选、约束、最短路、预约冲突、经验池、Q 更新和完整计划。
可编辑源	本手册同时提供 DOCX；PDF 用于打印，DOCX 用于继续批注与扩展。

第 1 章 先纠正分类：规划/非规划不是 AGV 算法总分类

一句话结论

“model-free（模型无关）与 model-based（模型式）”只是在强化学习内部回答‘智能体是否显式利用环境转移模型’。它不能覆盖 MILP、A*、CBS、GA、博弈、规则等全部 AGV 算法。

SB3 核心算法大多属于模型无关强化学习：PPO、A2C、DQN、SAC、TD3 等直接从交互数据学习价值或策略，并不先学习一个可供搜索的环境模型。模型式路线则先用已知/学习到的模型预测未来，再做 MPC、树搜索或虚拟滚动。

分类轴	关键问题	常见类别	AGV 示例
决策层	在决定什么？	配置/派工/联合排程/路径/冲突/充电/控制	PPO 派工、A*路径、CBS 冲突
求解范式	靠什么找到解？	规则/精确优化/图搜索/元启发式/RL/MARL/博弈/混合	MILP、GA、DQN、MADDPG
决策时机	什么时候重算？	离线/滚动时域/事件触发/在线	故障触发 D* Lite 重规划
系统架构	谁拥有信息和决策权？	集中式/分布式/分层/CTDE/市场	MAPPO 集中训练分散执行
不确定性	怎样处理未来不确定？	确定性/随机/鲁棒/DRO/风险敏感/自适应	CVaR 控制延误尾部风险

Q 表为什么没有出现在你原来的 SB3 项目里

Q-learning 把每个离散状态—动作的价值直接存进 Q 表；PPO 则把策略和价值存进神经网络参数。因此你的工程仍然有‘状态→动作→奖励→更新’，只是更新对象从 Q[state, action]变成了神经网络权重，SB3 又把梯度、批次、优势估计等细节封装进 model.learn()。

比较	Q-learning	PPO（SB3）
记忆载体	Q 表	策略网络 + 价值网络参数

比较	Q-learning	PPO (SB3)
动作产生	ϵ -greedy: 探索/利用	从策略分布采样; 评估时可确定性输出
适合动作	小型离散动作	离散或连续动作
经验形式	单步 TD 更新	一段 rollout 后按小批次多轮更新
AGV 定位	理解原理和小基线	复杂状态下的常用工程方案之一

第 2 章 任何 AGV 算法都应放进同一个工程母框架

算法不是项目的全部。稳定复用的关键，是把业务问题、算法、执行安全和实验评价拆开。下面这条链是本手册与配套代码的主干。

顺序	模块	固定职责
1	ProblemDefinition	地图、车辆、任务、约束、目标和权重
2	StateBuilder	把仿真器/数据库/数字孪生数据变成统一状态
3	CandidateGenerator	只生成当前可能的车辆—任务—路径候选
4	ConstraintChecker / ActionMask	删掉容量、电量、冲突、前序等非法动作
5	ObjectiveEvaluator	统一计算距离、延误、能耗、拥堵、优先级
6	DecisionEngine	规则/MILP/GA/PPO/MARL 等只在此处替换
7	PathPlanner + ConflictResolver	生成路线并处理节点/边冲突
8	SafetyShield	执行前确定性复核，不把安全押给奖励函数
9	Executor / DigitalTwinAdapter	推进仿真或下发真实系统
10	Experience + Learning	需要学习的算法才启用经验与参数更新
11	Evaluator + ExperimentManager	多场景、多随机种子、基线与消融
最重要的边界 A*不负责派哪台车，PPO 不应绕过硬安全规则，YOLO/LSTM/GNN 不是完整调度器，数字孪生是状态与执行平台。混合算法的本质，是让不同模块各做擅长的事。		

统一决策接口

母接口：所有算法都翻译成 `decide()`

```
class DecisionEngine(ABC):
```

```
"""所有算法共同遵守的母接口。"""

name = "decision-engine"

def reset(self, problem: ProblemDefinition) -> None:
    """新实验开始时清理内部状态；无状态算法可不做任何事。"""

    @abstractmethod
    def decide(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidates: Sequence[CandidateDecision],
    ) -> CandidateDecision:
        raise NotImplementedError

    def update(self, feedback: Any) -> None:
        """在线学习或滚动优化可接收反馈；静态规则可忽略。"""
```

只要新算法遵守该接口，环境、路径、安全、经验记录和评估都不必推倒重写。这也是‘项目级复用’与‘复制一段算法代码’的区别。

第 3 章 AGV 算法全景：15 个家族及其固定母模型

下面的‘固定’是指结构稳定，不是参数或实现永远不变。你先认母循环，再看论文用了哪个编码、网络、搜索算子或奖励。

3.1 派工规则与启发式

代表算法	最近车、最早可用、最短加工时间、最早交期、组合优先级、瓶颈优先
固定母模型	候选生成 → 硬约束过滤 → 指标打分 → 排序选优
优先场景	秒级响应、业务规则明确、作为任何复杂算法的对照组和安全兜底
主要优势	速度快、可解释、易上线
主要边界	近视；权重难泛化；可能造成长期拥堵
复用模块	CandidateGenerator + ConstraintChecker + ObjectiveEvaluator + RuleDecisionEngine

3.1.1 本家族可复用母模块代码

```
class CandidateGenerator:
    """生成 AGV × 任务候选对；复杂项目可加入候选任务窗口或邻域剪枝。"""

    def generate(self, snapshot: SchedulingSnapshot) -> list[CandidateDecision]:
        agvs = [a for a in snapshot.agvs if not a.busy and not a.faulted]
        tasks = [t for t in snapshot.tasks if not t.assigned]
        return [CandidateDecision(a.agv_id, t.task_id) for a in agvs for t in tasks]

class ConstraintChecker:
    """硬约束过滤器：违反即不能执行，不应只靠负奖励碰运气。"""

    def is_feasible(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidate: CandidateDecision,
    ) -> tuple[bool, str]:
        agv = snapshot.get_agv(candidate.agv_id)
        task = snapshot.get_task(candidate.task_id)
        if agv.busy or agv.faulted:
            return False, "AGV 忙碌或故障"
        if task.assigned:
            return False, "任务已分配"
        if task.load > agv.capacity:
            return False, "载荷超过容量"
        try:
            _, to_pickup = problem.graph.shortest_path(agv.node, task.pickup)
            _, to_dropoff = problem.graph.shortest_path(task.pickup, task.dropoff)
        except ValueError:
            return False, "路网不连通"
        predicted_battery = agv.battery - (to_pickup + to_dropoff) * problem.energy_per_distance
        if predicted_battery < problem.min_battery:
```

```
        return False, "预计执行后电量低于安全阈值"
    return True, "可行"

def filter(
    self,
    problem: ProblemDefinition,
    snapshot: SchedulingSnapshot,
    candidates: Iterable[CandidateDecision],
) -> list[CandidateDecision]:
    return [c for c in candidates if self.is_feasible(problem, snapshot, c)[0]]

class ObjectiveEvaluator:
    """统一目标函数：规则、优化和学习算法都用同一评价口径。"""

    def score(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidate: CandidateDecision,
    ) -> CandidateDecision:
        agv = snapshot.get_agv(candidate.agv_id)
        task = snapshot.get_task(candidate.task_id)
        _, empty_distance = problem.graph.shortest_path(agv.node, task.pickup)
        _, loaded_distance = problem.graph.shortest_path(task.pickup, task.dropoff)
        travel = empty_distance + loaded_distance
        energy = travel * problem.energy_per_distance
        tardiness = 0.0
        if task.due_time is not None:
            tardiness = max(0.0, snapshot.time + travel - task.due_time)
        w = problem.weights
        total = (
            w.get("travel", 1.0) * travel
            + w.get("energy", 0.0) * energy
            + w.get("tardiness", 0.0) * tardiness
            - w.get("priority", 0.0) * task.priority
        )
        reason = (
            f"空驶={empty_distance:.1f}, 载货={loaded_distance:.1f}, "
            f"能耗={energy:.1f}, 延误={tardiness:.1f}, 优先级={task.priority:.1f}"
        )
        return replace(candidate, score=total, explanation=reason)

class RuleDecisionEngine(DecisionEngine):
    """可解释基线：计算所有候选分数并取最小值。"""

    name = "weighted-nearest-rule"

    def __init__(self, evaluator: ObjectiveEvaluator | None = None) -> None:
        self.evaluator = evaluator or ObjectiveEvaluator()

    def decide(self, problem, snapshot, candidates):
        if not candidates:
            raise RuntimeError("当前没有可行的 AGV-任务候选")
        scored = [self.evaluator.score(problem, snapshot, c) for c in candidates]
        return min(scored, key=lambda x: (x.score, x.agv_id, x.task_id))
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.2 数学规划与精确优化

代表算法	LP、MILP、MINLP、CP、CP-SAT、SAT/SMT、指派、网络流、动态规划、B&B/B&C、Benders、列生成、拉格朗日松弛
------	--

固定母模型	定义变量 → 目标函数 → 约束 → 求解 → 解码与可行性检查
优先场景	中小规模离线排程、滚动时域优化、需要最优性界和严谨约束
主要优势	能表达复杂约束；解的含义清楚
主要边界	规模扩大时计算变慢；建模与求解器要求高
复用模块	ProblemDefinition + ExactSolverAdapter + ConstraintChecker

3.2.1 本家族可复用母模块代码

```
class ExactSolverAdapter(DecisionEngine):
    """
    MILP/CP-SAT/网络流等精确优化的适配母类。

    子类只需要把统一问题翻译给求解器，并将解翻译回 CandidateDecision。
    真实项目可分别实现 OrToolsCPSATEngine、GurobiMILPEngine 等。
    """

    name = "exact-solver-adapter"

    @abstractmethod
    def build_model(self, problem, snapshot, candidates) -> Any:
        raise NotImplementedError

    @abstractmethod
    def solve_model(self, model: Any) -> Any:
        raise NotImplementedError

    @abstractmethod
    def decode_solution(self, raw_solution: Any) -> CandidateDecision:
        raise NotImplementedError

    def decide(self, problem, snapshot, candidates):
        model = self.build_model(problem, snapshot, candidates)
        return self.decode_solution(self.solve_model(model))
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.3 不确定性与多目标优化

代表算法	随机规划、鲁棒优化、分布鲁棒优化、机会约束、CVaR、加权和、 ϵ -constraint、Pareto、NSGA-II/III、MOEA/D
固定母模型	构造随机场景/不确定集/多个目标 → 求解 → 风险或 Pareto 选择
优先场景	订单到达、加工时间、故障、能耗不确定；效率—能耗—延误多目标

主要优势	能把风险和折中显式写出来
主要边界	数据与分布假设敏感；输出需要二次决策
复用模块	ObjectiveEvaluator + ExperimentManager + Solver/Metaheuristic

3.3.1 本家族可复用母模块代码

```

class RiskEvaluator:
    """随机/鲁棒/风险敏感优化共用的场景代价聚合器。"""

    def aggregate(
        self,
        scenario_costs: Sequence[float],
        mode: str = "mean",
        alpha: float = 0.9,
    ) -> float:
        if not scenario_costs:
            raise ValueError("scenario_costs 不能为空")
        costs = sorted(float(x) for x in scenario_costs)
        if mode == "mean":
            return sum(costs) / len(costs)
        if mode == "worst":
            return costs[-1]
        if mode == "cvar":
            if not 0 <= alpha < 1:
                raise ValueError("CVaR alpha 必须位于 [0, 1)")
            tail_size = max(1, math.ceil((1.0 - alpha) * len(costs)))
            tail = costs[-tail_size:]
            return sum(tail) / len(tail)

        raise ValueError("mode 只支持 mean、worst 或 cvar")

class MultiObjectiveEvaluator:
    """多目标优化的 Pareto 前沿母模块。"""

    def __init__(self, minimize: Sequence[str] = (), maximize: Sequence[str] = ()) -> None:
        if not minimize and not maximize:
            raise ValueError("至少指定一个最小化或最大化目标")
        self.minimize = tuple(minimize)
        self.maximize = tuple(maximize)

    def dominates(self, left: dict[str, Any], right: dict[str, Any]) -> bool:
        no_worse = True
        strictly_better = False
        for name in self.minimize:
            no_worse &= left[name] <= right[name]
            strictly_better |= left[name] < right[name]
        for name in self.maximize:
            no_worse &= left[name] >= right[name]
            strictly_better |= left[name] > right[name]
        return bool(no_worse and strictly_better)

    def pareto_front(self, records: Sequence[dict[str, Any]]) -> list[dict[str, Any]]:
        return [
            record
            for i, record in enumerate(records)
            if not any(
                i != j and self.dominates(other, record)
                for j, other in enumerate(records)
            )
        ]

```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.4 单车图搜索与增量规划

代表算法	BFS、UCS、Dijkstra、A*、D*、D* Lite、LPA*、JPS、Theta*、SIPP、时间扩展网络、PRM、RRT*
固定母模型	开放集/树 → 扩展邻居 → 更新代价 → 终点回溯；动态算法复用旧搜索结果
优先场景	路网最短路、动态障碍重规划、连续空间运动规划
主要优势	结构清楚、可验证、常有完备性/最优性保证
主要边界	单车最优不等于车队最优；需另做冲突处理
复用模块	WeightedGraph + PathPlanner

3.4.1 本家族可复用母模块代码

```
class WeightedGraph:
    """轻量有权图；可替换为真实路网、NetworkX 或数字孪生地图服务。"""
    def __init__(self) -> None:
        self._adj: dict[str, list[tuple[str, float]]] = defaultdict(list)
    def add_edge(self, a: str, b: str, cost: float = 1.0, bidirectional: bool = True) -> None:
        if cost <= 0:
            raise ValueError("边权必须大于0")
        self._adj[a].append((b, float(cost)))
        self._adj.setdefault(b, [])
        if bidirectional:
            self._adj[b].append((a, float(cost)))
    def neighbors(self, node: str) -> tuple[tuple[str, float], ...]:
        return tuple(self._adj.get(node, ()))
    def shortest_path(
        self,
        start: str,
        goal: str,
        heuristic: Callable[[str, str], float] | None = None,
    ) -> tuple[list[str], float]:
        """Dijkstra; 传入可采纳heuristic 时就是A* 的母模板。"""
        if start == goal:
            return [start], 0.0
        h = heuristic or (lambda _a, _b: 0.0)
        frontier: list[tuple[float, float, str]] = [(h(start, goal), 0.0, start)]
        previous: dict[str, str | None] = {start: None}
        best_g: dict[str, float] = {start: 0.0}
        while frontier:
            _f, g, node = heappop(frontier)
            if g != best_g.get(node):
                continue
            if node == goal:
                path: list[str] = []
                cursor: str | None = goal
                while cursor is not None:
                    path.append(cursor)
                    cursor = previous[cursor]
                return list(reversed(path)), g
            for nxt, edge_cost in self.neighbors(node):
                new_g = g + edge_cost
                if new_g < best_g.get(nxt, math.inf):
                    best_g[nxt] = new_g
```

```

        previous[nxt] = node
        heappush(frontier, (new_g + h(nxt, goal), new_g, nxt))
        raise ValueError(f"节点 {start!r} 与 {goal!r} 之间不存在可行路径")

class PathPlanner:
    """路径规划统一入口；可替换为 A*、SIPP、CBS、D* Lite 或外部服务。"""

    def plan(self, graph: WeightedGraph, start: str, goal: str) -> tuple[list[str], float]:
        return graph.shortest_path(start, goal)
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.5 多智能体路径与冲突消解（MAPF）

代表算法	优先级规划、预约表、Cooperative A*、WHCA*、CBS、ECBS、M*、Push-and-Swap、PIBT
固定母模型	先生成路径 → 检测节点/边冲突 → 加约束、等待或优先级 → 局部/全局重规划
优先场景	多 AGV 会车、交叉口、窄巷道、死锁与拥堵
主要优势	直接针对多车冲突；可与上层调度解耦
主要边界	规模大时冲突树或联合空间膨胀；优先级会偏置
复用模块	ReservationTable + ConflictResolver + SafetyShield + PathPlanner

3.5.1 本家族可复用母模块代码

```

class ReservationTable:
    """简化时空预约表：检查节点冲突和对向边冲突。"""

    def __init__(self) -> None:
        self.nodes: dict[tuple[int, str], str] = {}
        self.edges: dict[tuple[int, str, str], str] = {}

    def is_free(self, path: Sequence[str], start_time: int, owner: str = "candidate") -> bool:
        for i, node in enumerate(path):
            t = start_time + i
            occupant = self.nodes.get((t, node))
            if occupant is not None and occupant != owner:
                return False
            if i > 0:
                prev = path[i - 1]
                opposite = self.edges.get((t, node, prev))
                if opposite is not None and opposite != owner:
                    return False
        return True

    def reserve(self, path: Sequence[str], start_time: int, owner: str) -> None:
        for i, node in enumerate(path):
            t = start_time + i
            self.nodes[(t, node)] = owner
            if i > 0:
                self.edges[(t, path[i - 1], node)] = owner
```

```
def resolve(
    self,
    path: Sequence[str],
    start_time: int,
    max_wait_steps: int = 50,
    owner: str = "candidate",
) -> tuple[list[str], int]:
    if not path:
        raise ValueError("路径不能为空")
    for waits in range(max_wait_steps + 1):
        delayed = [path[0]] * waits + list(path)
        if self.is_free(delayed, start_time, owner):
            return delayed, waits
    raise RuntimeError("在最大等待步数内无法消解路径冲突")

class ConflictResolver(ReservationTable):
    """预约表的工程封装；以后可由 CBS、ECBS 或 PIBT 实现同一职责。"""

    def resolve_and_reserve(
        self,
        path: Sequence[str],
        start_time: int,
        owner: str,
        max_wait_steps: int = 50,
    ) -> tuple[list[str], int]:
        repaired, waits = self.resolve(path, start_time, max_wait_steps, owner)
        self.reserve(repaired, start_time, owner)
        return repaired, waits
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.6 元启发式与超启发式

代表算法	GA、DE、PSO、ACO、SA、TS、VNS、ALNS、ILS、GRASP、Memetic、ABC、FOA、FPA、GWO、WOA、TLBO
固定母模型	编码 → 初始化种群/解 → 评价 → 搜索算子 → 修复/接受 → 终止
优先场景	柔性车间、任务—车辆—路径联合组合优化、复杂非线性目标
主要优势	对模型形式要求较低；能得到较好可行解
主要边界	无最优性保证；参数和编码决定上限；算法名多但母循环相似
复用模块	MetaheuristicEngine + encode/decode/repair/fitness/operators

3.6.1 本家族可复用母模块代码

```
class MetaheuristicEngine:
```

```
"""GA/PSO/ACO/VNS/ALNS 等可共用的“解—评价—扰动—接受”母循环。"""

def __init__(self, iterations: int = 100, seed: int = 0) -> None:
    self.iterations = iterations
    self.rng = random.Random(seed)

    def optimize(
        self,
        initial_solution: Any,
        objective: Callable[[Any], float],
        neighbor: Callable[[Any, random.Random], Any],
        repair: Callable[[Any], Any] | None = None,
        accept: Callable[[float, float, int], bool] | None = None,
    ) -> tuple[Any, float]:
        repair_fn = repair or (lambda x: x)
        accept_fn = accept or (lambda new, old, _i: new <= old)
        current = repair_fn(initial_solution)
        current_score = objective(current)
        best, best_score = current, current_score
        for i in range(self.iterations):
            trial = repair_fn(neighbor(current, self.rng))
            trial_score = objective(trial)
            if accept_fn(trial_score, current_score, i):
                current, current_score = trial, trial_score
            if trial_score < best_score:
                best, best_score = trial, trial_score
        return best, best_score
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.7 表格强化学习

代表算法	Q-learning、SARSA、Expected SARSA、Dyna-Q
固定母模型	状态—动作交互 → 奖励 → TD 目标 → Q 值更新 → ϵ -greedy 探索衰减
优先场景	小离散状态、教学、小规模调度基线、可解释策略表
主要优势	最容易看懂‘奖励如何改变行为’
主要边界	状态组合爆炸；复杂多车不适合 Q 表
复用模块	TabularQLearner + ExperienceModule

3.7.1 本家族可复用母模块代码

```
class TabularQLearner:
    """Q-learning 的最小可运行内核；用于理解，不是复杂 AGV 的最终算法。"""

    def __init__(self, alpha: float = 0.1, gamma: float = 0.95, seed: int = 0) -> None:
        self.alpha = alpha
        self.gamma = gamma
```

```
self.q: dict[tuple[Hashable, Hashable], float] = defaultdict(float)
self.rng = random.Random(seed)

def get_q(self, state: Hashable, action: Hashable) -> float:
    return self.q[(state, action)]

def set_q(self, state: Hashable, action: Hashable, value: float) -> None:
    self.q[(state, action)] = float(value)

def choose_action(
    self, state: Hashable, actions: Sequence[Hashable], epsilon: float
) -> Hashable:
    if not actions:
        raise ValueError("actions 不能为空")

    if self.rng.random() < epsilon: # 探索
        return self.rng.choice(list(actions))

    return max(actions, key=lambda a: (self.get_q(state, a), str(a))) # 利用

def update(
    self,
    state: Hashable,
    action: Hashable,
    reward: float,
    next_state: Hashable,
    next_actions: Sequence[Hashable],
    done: bool = False,
) -> float:
    old_q = self.get_q(state, action)
    future = 0.0 if done or not next_actions else max(self.get_q(next_state, a) for a in next_actions)
    target = reward + self.gamma * future
    td_error = target - old_q
    self.set_q(state, action, old_q + self.alpha * td_error)
    return td_error
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.8 深度价值强化学习

代表算法	DQN、Double DQN、Dueling DQN、PER、NoisyNet、C51、QR-DQN、Rainbow
固定母模型	回放采样 → TD 目标 → Q 网络更新 → 目标网络同步 → ϵ -greedy
优先场景	离散动作的任务分配、工序选择、路径节点选择
主要优势	用神经网络替代 Q 表，可处理高维状态
主要边界	仍受离散动作和动作数量影响；训练稳定性需控制
复用模块	ReplayBuffer + RLDecisionAdapter + ActionMask

3.8.1 本家族可复用母模块代码

```
class ReplayBuffer(ExperienceModule):
    """DQN/DDPG/TD3/SAC 常用的离策略经验回放。"""

    def __init__(self, capacity: int, seed: int = 0) -> None:
        if capacity <= 0:
```

```
        raise ValueError("capacity 必须大于 0")
        self._items: deque[Transition] = deque(maxlen=capacity)
        self._rng = random.Random(seed)

    def add(self, transition: Transition) -> None:
        self._items.append(transition)

    def sample(self, batch_size: int) -> list[Transition]:
        if batch_size > len(self._items):
            raise ValueError("样本数不足")
        return self._rng.sample(list(self._items), batch_size)

    def __len__(self) -> int:
        return len(self._items)

class RLDecisionAdapter(DecisionEngine):
    """
    PPO/DQN/SAC 等模型无关强化学习的统一适配器。

    encoder: SchedulingSnapshot -> 算法需要的 observation
    predictor: observation, valid_action_ids -> action_id
    decoder: action_id, candidates -> CandidateDecision
    """

    name = "rl-policy-adapter"

    def __init__(
        self,
        encoder: Callable[[SchedulingSnapshot], Any],
        predictor: Callable[[Any, list[int]], int],
    ) -> None:
        self.encoder = encoder
        self.predictor = predictor

    def decide(self, problem, snapshot, candidates):
        if not candidates:
            raise RuntimeError("没有可行动作")
        observation = self.encoder(snapshot)
        valid_action_ids = list(range(len(candidates))) # action mask 的最小形式
        action_id = self.predictor(observation, valid_action_ids)
        if action_id not in valid_action_ids:
            raise RuntimeError(f"策略返回了非法动作 {action_id}")
        return candidates[action_id]
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.9 策略梯度与 Actor-Critic

代表算法	REINFORCE、A2C/A3C、TRPO、PPO、DDPG、TD3、SAC
固定母模型	采样轨迹 → 估计回报/优势 → 更新策略；Actor 选动作，Critic 评估
优先场景	PPO 适合离散/连续综合策略；SAC/TD3 偏连续控制；复杂调度常用 PPO
主要优势	策略可随机、可处理连续动作；PPO 工程生态成熟
主要边界	样本效率与奖励设计敏感；训练成功不等于约束满足

复用模块	RolloutBuffer/ReplayBuffer + RLDecisionAdapter + SafetyShield
------	---

3.9.1 本家族可复用母模块代码

```
class RolloutBuffer(ExperienceModule):
    """PP0/A2C 常用的同策略轨迹缓存；训练一次后通常清空。"""

    def __init__(self) -> None:
        self._items: list[Transition] = []

    def add(self, transition: Transition) -> None:
        self._items.append(transition)

    def consume(self) -> list[Transition]:
        batch, self._items = self._items, []
        return batch

    def __len__(self) -> int:
        return len(self._items)
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.10 模型式强化学习与规划学习

代表算法	Dyna-Q、学习动力学+MPC、PETS、MBPO、Dreamer、MuZero
固定母模型	学习/已知环境模型 → 虚拟滚动预测未来 → 搜索或更新策略 → 真环境校正
优先场景	真实试错昂贵、数字孪生可用、需要预测拥堵/故障后果
主要优势	更省真实样本；能预演未来
主要边界	模型误差会累积；计算和系统复杂度更高
复用模块	DigitalTwinAdapter + LearnedModel + Planner + HybridCoordinator

3.10.1 本家族可复用母模块代码

```
class ModelBasedPlanner:
    """
    模型式强化学习/学习模型+MPC 的通用短视域规划器。

    transition_model 预测下一状态，reward_model 评价一步结果；
    planner 在模型中向前滚动，而不是只凭当前策略直接出动作。
    """

    def __init__(
```

```
self,
    action_provider: Callable[[Any], Sequence[Any]],
    transition_model: Callable[[Any, Any], Any],
    reward_model: Callable[[Any, Any, Any], float],
    horizon: int = 3,
    discount: float = 0.99,
    beam_width: int = 16,
) -> None:
    if horizon <= 0 or beam_width <= 0:
        raise ValueError("horizon 和 beam_width 必须大于 0")
    self.action_provider = action_provider
    self.transition_model = transition_model
    self.reward_model = reward_model
    self.horizon = horizon
    self.discount = discount
    self.beam_width = beam_width

def plan(self, initial_state: Any) -> tuple[Any, float, list[Any]]:
    beams: list[tuple[float, Any, list[Any]]] = [(0.0, initial_state, [])]
    for depth in range(self.horizon):
        expanded: list[tuple[float, Any, list[Any]]] = []
        for score, state, sequence in beams:
            for action in self.action_provider(state):
                next_state = self.transition_model(state, action)
                reward = self.reward_model(state, action, next_state)
                total = score + (self.discount**depth) * reward
                expanded.append((total, next_state, sequence + [action]))
        if not expanded:
            break
        expanded.sort(key=lambda item: item[0], reverse=True)
        beams = expanded[: self.beam_width]
    if not beams or not beams[0][2]:
        raise RuntimeError("模型规划器没有找到可行动作")
    best_score, _state, sequence = beams[0]
    return sequence[0], best_score, sequence
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.11 多智能体强化学习（MARL）

代表算法	IQL、VDN、QMIX、QTRAN、COMA、MADDPG、MAPPO、MASAC、CTDE
固定母模型	各车局部观测 → 联合交互 → 集中训练/分散执行 → 信用分配
优先场景	多 AGV 协作、局部通信、分布式执行、车队能量均衡
主要优势	能建模协作与竞争；执行可分散
主要边界	非平稳、信用分配和扩展性困难；必须配安全层
复用模块	MARLDecisionAdapter + joint observation/action + SafetyShield

3.11.1 本家族可复用母模块代码

```
class MARLDecisionAdapter(DecisionEngine):
    """IQL/QMIX/MADDPG/MAPPO 等多智能体算法的联合动作适配母类。"""

    name = "marl-adapter"

    @abstractmethod
    def decide_jointly(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot
    ) -> dict[str, Any]:
        raise NotImplementedError

    def decide(self, problem, snapshot, candidates):
        joint = self.decide_jointly(problem, snapshot)
        agv_id = str(joint["agv_id"])
        task_id = str(joint["task_id"])
        match = next((c for c in candidates if c.agv_id == agv_id and c.task_id == task_id), None)
        if match is None:
            raise RuntimeError("多智能体联合动作不满足当前约束")
        return match
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.12 分层强化学习（HRL）

代表算法	Options、MAXQ、Feudal、Goal-conditioned、Manager-Worker
固定母模型	高层选择任务/子目标 → 低层执行路径/动作 → 分层奖励和终止条件
优先场景	派工与路径不同时间尺度、复杂任务分解
主要优势	与 AGV ‘上层派工一下层路径’ 天然对应
主要边界	层间目标可能不一致；训练和调试更难
复用模块	HybridCoordinator + high_policy + low_policy

3.12.1 本家族可复用母模块代码

```
class HierarchicalController:
    """分层强化学习/两阶段调度的最小母接口。"""

    def __init__(
        self,
        high_level_policy: Callable[[Any], Any],
        low_level_policy: Callable[[Any, Any], Any],
    ) -> None:
        self.high_level_policy = high_level_policy
        self.low_level_policy = low_level_policy
```

```
def decide(self, state: Any) -> tuple[Any, Any]:
    subgoal = self.high_level_policy(state)
    primitive_action = self.low_level_policy(state, subgoal)
    return subgoal, primitive_action
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

判断提示：先问这类方法在当前论文中负责哪一层，再问它是否与规则、路径规划或安全模块组合。不要因为标题出现某个算法，就假定它承担全部任务。

3.13 安全、约束、离线与模仿学习

代表算法	Action Mask、Safety Shield、CMDP、拉格朗日 PPO、CPO、BC、GAIL、BCQ、CQL、IQL
固定母模型	约束/历史日志 → 安全动作集或离线训练 → 部署前验证 → 运行时保护
优先场景	真实车不能冒险、已有专家规则或历史调度日志
主要优势	降低非法动作和在线探索风险
主要边界	离线数据覆盖不足会外推失败；安全证明仍需工程规则
复用模块	ConstraintChecker + ActionMask + SafetyShield + ReplayBuffer

3.13.1 本家族可复用母模块代码

```
@dataclass(frozen=True)
class ActionMask:
    """把硬约束转换成策略能使用的合法动作掩码。"""

    allowed: tuple[bool, ...]

    @classmethod
    def from_keys(
        cls,
        all_action_keys: Sequence[Hashable],
        feasible_action_keys: set[Hashable],
    ) -> "ActionMask":
        return cls(tuple(key in feasible_action_keys for key in all_action_keys))

    @classmethod
    def from_candidates(
        cls,
        all_candidates: Sequence[CandidateDecision],
        feasible_candidates: Sequence[CandidateDecision],
    ) -> "ActionMask":
        feasible = {(x.agv_id, x.task_id) for x in feasible_candidates}
        keys = [(x.agv_id, x.task_id) for x in all_candidates]
        return cls.from_keys(keys, feasible)

    def legal_indices(self) -> list[int]:
        return [i for i, allowed in enumerate(self.allowed) if allowed]

    def validate(self, action_index: int) -> None:
        if action_index < 0 or action_index >= len(self.allowed):
```

```
        raise ValueError(f"动作索引 {action_index} 越界")
    if not self.allowed[action_index]:
        raise ValueError(f"动作索引 {action_index} 被硬约束屏蔽")

class SafetyShield:
    """安全盾：最终动作执行前再做确定性校验。"""

    def validate(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
    ) -> tuple[bool, str]:
        agv = snapshot.get_agv(plan.agv_id)
        task = snapshot.get_task(plan.task_id)
        if agv.faulted or agv.busy:
            return False, "AGV 状态不可执行"
        if task.load > agv.capacity:
            return False, "载荷超限"
        remaining = agv.battery - plan.distance * problem.energy_per_distance
        if remaining < problem.min_battery:
            return False, "执行后电量越过安全线"
        if plan.full_path[0] != agv.node or plan.full_path[-1] != task.dropoff:
            return False, "路径起终点与任务不一致"
        return True, "安全校验通过"
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.14 博弈、拍卖与市场机制

代表算法	非合作博弈、势博弈、演化博弈、Stackelberg、拍卖、合同网
固定母模型	定义参与者/策略/效用 → 最优响应或竞价 → 均衡/分配 → 执行
优先场景	分布式资源竞争、订单竞价、能源和拥堵外部性
主要优势	机制解释清楚，适合分布式协调
主要边界	均衡不必等于系统全局最优；效用设计决定结果
复用模块	GameCoordinator + ObjectiveEvaluator + MARLDecisionAdapter

3.14.1 本家族可复用母模块代码

```
class GameCoordinator:
    """势博弈/非合作博弈/拍卖前的通用最优响应母循环。"""

    def __init__(
        self,
        players: Sequence[Hashable],
        action_provider: Callable[[Hashable, Any], Sequence[Any]],
        utility: Callable[[Hashable, Any, dict[Hashable, Any], Any], float],
    ) -> None:
        self.players = tuple(players)
        self.action_provider = action_provider
        self.utility = utility
```

```
def best_response(
    self,
    state: Any,
    initial: dict[Hashable, Any],
    rounds: int = 20,
) -> dict[Hashable, Any]:
    joint = dict(initial)
    for _ in range(rounds):
        changed = False
        for player in self.players:
            actions = list(self.action_provider(player, state))
            if not actions:
                continue
            best = max(
                actions,
                key=lambda action: (
                    self.utility(player, action, joint, state),
                    str(action),
                ),
            )
            changed |= joint.get(player) != best
            joint[player] = best
        if not changed:
            break
    return joint
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

3.15 感知、预测与表征学习

代表算法	YOLO、CNN、LSTM/GRU、Transformer、GNN、时空图网络
固定母模型	传感器/历史/图 → 检测、预测或编码 → 将特征交给优化器/策略
优先场景	行人避障、拥堵预测、图路网编码、订单时序预测
主要优势	提升状态质量与泛化能力
主要边界	它们通常不是完整调度器；必须连接决策与安全模块
复用模块	StateBuilder + Predictor/Encoder + DecisionEngine

3.15.1 本家族可复用母模块代码

```
class StateBuilder:
    """把数据库/数字孪生/仿真器原始数据整理为统一快照。"""

    def build(
        self,
        time: int,
        agvs: Iterable[AGV],
        tasks: Iterable[Task],
        metadata: dict[str, Any] | None = None,
    ) -> SchedulingSnapshot:
        return SchedulingSnapshot(time, tuple(agvs), tuple(tasks), metadata or {})

class FeaturePipeline:
    """
```

感知/预测/表征母模块。

YOLO、LSTM、Transformer、GNN 等通常只是流水线中的一个变换，
最终输出仍要交给 DecisionEngine，而不是直接绕过约束和安全层。
"""

```
def __init__(self, transforms: Sequence[Callable[[Any], Any]]) -> None:
    self.transforms = tuple(transforms)

def transform(self, raw_input: Any) -> Any:
    value = raw_input
    for transform in self.transforms:
        value = transform(value)
    return value
```

使用方法：保持输入、输出和固定职责不变；新论文或新算法只替换类内部的求解器、网络、算子、预测器或评价细节。

第 4 章 按实际问题选算法，而不是按流行度选算法

问题特征	首选家族	常见组合	注意
需要快速上线并可解释	规则 + A*/Dijkstra + 预约表	MILP/元启发式作离线对照	先建立可靠基线和评估闭环
中小规模、约束严、要最优性界	MILP/CP-SAT/网络流	滚动时域 + 启发式热启动	计算时间必须纳入指标
大型柔性车间联合排程	GA/VNS/ALNS/混合元启发式	数学模型负责约束/下界	编码、修复算子比名字更重要
离散任务分配、状态高维	DQN 系或 PPO	Action Mask + 规则兜底	DQN 受动作数量影响
连续速度/能量控制	SAC/TD3/PPO	MPC 或安全盾	调度层与控制层不要混成一个动作
多 AGV 分散执行	MAPPO/QMIX/MADDPG 等 MARL	CTDE + 冲突解析 + 安全盾	先证明集中式基线，再上 MARL
上层派工、下层路径	分层架构/HRL	规则/MILP/PPO + A*/CBS	这是 AGV 最常见的合理混合
真实试错昂贵且有数字孪生	模型式 RL/MPC/仿真训练	离线 RL + 域随机化	重点防模型偏差和仿真到现实差距
有人车混行	预测 + 安全规划/PPO	YOLO/LSTM + SIPP/MPC/Shield	感知输出不能直接等于最终动作
已有大量历史调度日志	模仿/离线 RL	BC 预训练 + 在线小步微调	检查数据覆盖和行为策略偏差

算法选择七问

问题	如果答不出来，先别换算法
1. 决策对象是什么？	任务—车辆、工序—机器、路径节点、速度、充电，还是联合动作？
2. 动作是否合法？	容量、电量、工序前序、冲突、死锁怎样被硬约束？
3. 规模和时限？	车辆/任务/节点数量，以及每次决策允许多少毫秒/秒？
4. 不确定来自哪里？	订单、加工时长、故障、行人、通信还是地图变化？
5. 基线是谁？	规则、MILP 下界、A*/CBS、GA/ALNS 中至少一个强基线。
6. 成功指标？	完工期、延误、空驶、能耗、吞吐、冲突、计算时长和鲁棒性。
7. 部署边界？	集中式还是分散式？仿真还是实车？异常时如何兜底？

第 5 章 论文中的算法为什么会‘杂糅’

混合不是把名字堆在一起，而是把不同职责分给不同模块。真正可复用的是‘角色组合模式’。

混合模式	职责分工	代表
数学模型 + 元启发式	模型定义变量/约束/目标；元启发式在大空间搜可行近优解	MILP + GAIVNS、模型 + IFPA
上层派工 + 下层路径	规则/MILP/RL 负责车辆任务；A*/SIPP/CBS 负责安全路线	两阶段任务路径、2D-PPO
RL 调元启发式	RL 不直接输出车辆动作，而是选择局部搜索算子或参数	Q-learning + FOA
RL + 工程启发式	策略提高适应性；Action Mask/防绕行/安全盾保证底线	PPO + 防绕行
经典规划 + RL + 冲突器	A*找路，RL 学习难以手工设计的决策，CBS 消冲突	数字孪生 + A* + DQN + CBS
博弈 + MARL	博弈定义效用和交互；MARL 学习响应策略	势博弈 + MADDPG
感知 + 预测 + RL	YOLO 检测，LSTM 预测，PPO 决策	人车混行避障
多策略/多智能体分解	不同代理分别做订单、车辆、机器或不同目标	DMA-DDQN、MSPPO
数字孪生 + 求解器	孪生体给实时状态/验证；算法求解；执行反馈再更新	动态在线调度

混合算法的模块识别法

读到一种混合算法时，依次标出：状态由谁构造、候选由谁生成、约束由谁保证、上层由谁决策、下层由谁规划、冲突由谁解决、参数由谁学习、结果由谁评价。只要能填入这些槽位，就能把论文方法迁移进母框架。论文题录和 Notion 链接已经移至独立《AGV 算法论文映射与文献索引》。

第 6 章 V2 可复用母模块：从‘三模块’升级为工程闭环

你原来的决策模块、经验模块、学习模块是强化学习工程的核心，但还不足以覆盖 MILP、A*、CBS、元启发式、数字孪生和安全执行。V2 保留原理解方式，在外围补齐问题定义、约束、路径、冲突、安全和统一评估。

模块	复用属性	固定职责	服务对象
ProblemDefinition	稳定	地图、车辆、任务、目标、硬约束	所有算法
StateBuilder	项目替换	原始数据→统一状态	仿真/数字孪生/实车
CandidateGenerator	常替换	缩小可行动作集合	规则/优化/RL
ConstraintChecker	必须	容量、电量、前序、禁行等硬约束	所有算法
ActionMask	RL 必需	把约束转换为策略可见的合法动作掩码	DQN/PPO/MARL
ObjectiveEvaluator	必须	统一评价口径	规则/优化/学习/实验
DecisionEngine	核心可换	输出车辆—任务等决策	Rule/MILP/GA/RL/MARL
ExperienceModule	按需	Replay 或 Rollout 经验	学习算法
LearningModule	按需	从经验更新参数	Q/DQN/PPO/MARL
ExactSolverAdapter	可插拔	翻译数学模型与求解器解	MILP/CP-SAT
MetaheuristicEngine	可插拔	编码/评价/算子/修复/接受	GA/VNS/ALNS 等
PathPlanner	通常必须	单车/增量/时空路径	A*/SIPP/D* Lite
ConflictResolver	多车必须	节点/边冲突和死锁	预约/CBS/PIBT
HybridCoordinator	工程核心	安排上层和下层算法调用顺序	两阶段/分层/混合
SafetyShield	部署必须	执行前最终拒绝非法动作	所有学习/启发式算法
DigitalTwinAdapter	项目接口	实时状态、仿真预演、计划下发	模型式/在线调度
Executor	项目接口	推进仿真或连接 WCS	所有算法
Evaluator	必须	统一记录性能、风险、时间	所有实验
ExperimentManager	必须	场景、随机种子、基线、消融	论文与选型

模块	复用属性	固定职责	服务对象
AlgorithmCatalog	学习/配置	算法家族、场景和模块索引	选型与教学

旧版三模块 → V2

旧认识	保留	V2 新增边界
决策模块	仍负责从状态输出动作	前面增加候选/约束/目标；后面增加路径/冲突/安全
经验模块	ReplayBuffer/RolloutBuffer 继续复用	只有学习算法需要；MILP/A*/规则不强行套经验池
学习模块	Q/DQN/PPO 更新仍在这里	与 DecisionEngine 解耦，便于训练和部署分离
环境/Wrapper/ Callback	继续作为仿真与训练外壳	补充 DigitalTwinAdapter、Evaluator 和 ExperimentManager

第 7 章 工程总装母模板（与完整.py 一致）

第三章已经把 15 类算法各自的母模块就地列出。本章只保留跨算法共享的总装代码，说明怎样把决策、路径、冲突、安全、执行和评估串成一个闭环。

7.1 数据模型与项目定义

```
@dataclass(frozen=True)
class AGV:
    """一台 AGV 在某个决策时刻的状态。"""

    agv_id: str
    node: str
    battery: float
    capacity: float
    busy: bool = False
    faulted: bool = False

@dataclass(frozen=True)
class Task:
    """一个运输任务。priority 越大，越希望优先处理。"""

    task_id: str
    pickup: str
    dropoff: str
    load: float
    priority: float = 1.0
    assigned: bool = False
    due_time: int | None = None

@dataclass(frozen=True)
class SchedulingSnapshot:
    """一次决策看到的完整快照；对应强化学习中的 observation/state。"""

    time: int
    agvs: tuple[AGV, ...]
    tasks: tuple[Task, ...]
    metadata: dict[str, Any] = field(default_factory=dict)

    def get_agv(self, agv_id: str) -> AGV:
        return next(x for x in self.agvs if x.agv_id == agv_id)

    def get_task(self, task_id: str) -> Task:
        return next(x for x in self.tasks if x.task_id == task_id)

@dataclass(frozen=True)
class ProblemDefinition:
    """项目级固定配置。新的 AGV 项目主要替换这里和状态构造器。"""

    graph: WeightedGraph
    min_battery: float = 15.0
    energy_per_distance: float = 1.0
    max_wait_steps: int = 50
    weights: dict[str, float] = field(
        default_factory=lambda: {
            "travel": 1.0,
            "tardiness": 2.0,
            "energy": 0.2,
            "priority": 1.0,
        }
    )
)
```

7.2 统一决策接口

```
class DecisionEngine(ABC):
    """所有算法共同遵守的母接口。"""

    name = "decision-engine"

    def reset(self, problem: ProblemDefinition) -> None:
        """新实验开始时清理内部状态；无状态算法可不做任何事。"""

    @abstractmethod
    def decide(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        candidates: Sequence[CandidateDecision],
    ) -> CandidateDecision:
        raise NotImplementedError

    def update(self, feedback: Any) -> None:
        """在线学习或滚动优化可接收反馈；静态规则可忽略。"""
```

7.3 混合协调器

```
@dataclass(frozen=True)
class ExecutionPlan:
    agv_id: str
    task_id: str
    to_pickup: list[str]
    to_dropoff: list[str]
    distance: float
    wait_steps: int = 0
    safe: bool = False
    explanation: str = ""

    @property
    def full_path(self) -> list[str]:
        if not self.to_pickup:
            return list(self.to_dropoff)
        return list(self.to_pickup) + list(self.to_dropoff[1:])

class HybridCoordinator:
    """上层派工 + 下层路径 + 冲突处理 + 安全盾的可运行组合。"""

    def __init__(
        self,
        decision_engine: DecisionEngine,
        candidate_generator: CandidateGenerator | None = None,
        constraint_checker: ConstraintChecker | None = None,
        path_planner: PathPlanner | None = None,
        conflict_resolver: ReservationTable | None = None,
        safety_shield: SafetyShield | None = None,
    ) -> None:
        self.decision_engine = decision_engine
        self.candidate_generator = candidate_generator or CandidateGenerator()
        self.constraint_checker = constraint_checker or ConstraintChecker()
        self.path_planner = path_planner or PathPlanner()
        self.conflict_resolver = conflict_resolver or ConflictResolver()
        self.safety_shield = safety_shield or SafetyShield()

    def plan_one(self, problem: ProblemDefinition, snapshot: SchedulingSnapshot) -> ExecutionPlan:
        raw = self.candidate_generator.generate(snapshot)
        feasible = self.constraint_checker.filter(problem, snapshot, raw)
        decision = self.decision_engine.decide(problem, snapshot, feasible)
        agv = snapshot.get_agv(decision.agv_id)
        task = snapshot.get_task(decision.task_id)

        pickup_path, pickup_cost = self.path_planner.plan(problem.graph, agv.node, task.pickup)
        dropoff_path, dropoff_cost = self.path_planner.plan(problem.graph, task.pickup, task.dropoff)
        original_full = pickup_path + dropoff_path[1:]
```

```

        resolved_full, waits = self.conflict_resolver.resolve(
            original_full,
            start_time=snapshot.time,
            max_wait_steps=problem.max_wait_steps,
            owner=agv.agv_id,
        )
        # 等待都加在起点，因此可按取货节点位置重新切分。
        pickup_end = max(i for i, node in enumerate(resolved_full) if node == task.pickup)
        to_pickup = resolved_full[: pickup_end + 1]
        to_dropoff = resolved_full[pickup_end:]
        plan = ExecutionPlan(
            agv_id=agv.agv_id,
            task_id=task.task_id,
            to_pickup=to_pickup,
            to_dropoff=to_dropoff,
            distance=pickup_cost + dropoff_cost,
            wait_steps=waits,
            explanation=decision.explanation,
        )
        safe, safety_reason = self.safety_shield.validate(problem, snapshot, plan)
        if not safe:
            raise RuntimeError(f"安全盾拒绝计划: {safety_reason}")
        plan = replace(plan, safe=True, explanation=f"{plan.explanation}; {safety_reason}")
        self.conflict_resolver.reserve(plan.full_path, snapshot.time, agv.agv_id)
        return plan

```

7.4 执行、评价与实验

```

class Executor:
    """教学用状态推进器；真实项目应替换为 WCS/仿真器/数字孪生接口。"""

    def apply(
        self, problem: ProblemDefinition, snapshot: SchedulingSnapshot, plan: ExecutionPlan
    ) -> SchedulingSnapshot:
        agvs = tuple(
            replace(
                a,
                node=snapshot.get_task(plan.task_id).dropoff,
                battery=a.battery - plan.distance * problem.energy_per_distance,
                busy=False,
            )
            if a.agv_id == plan.agv_id
            else a
            for a in snapshot.agvs
        )
        tasks = tuple(t for t in snapshot.tasks if t.task_id != plan.task_id)
        return SchedulingSnapshot(
            time=snapshot.time + len(plan.full_path) - 1,
            agvs=agvs,
            tasks=tasks,
            metadata=dict(snapshot.metadata),
        )

@dataclass
class EvaluationMetrics:
    completed_tasks: int = 0
    total_distance: float = 0.0
    total_wait_steps: int = 0
    safety_rejections: int = 0
    decision_count: int = 0

    @property
    def average_distance(self) -> float:
        return self.total_distance / self.completed_tasks if self.completed_tasks else 0.0

class Evaluator:
    def __init__(self) -> None:
        self.metrics = EvaluationMetrics()

    def record(self, plan: ExecutionPlan) -> None:
        self.metrics.completed_tasks += 1

```

```
        self.metrics.decision_count += 1
        self.metrics.total_distance += plan.distance
        self.metrics.total_wait_steps += plan.wait_steps

class ExperimentManager:
    """固定随机种子、算法、场景和评价指标，避免只看单次 reward。"""

    def __init__(self, coordinator: HybridCoordinator, evaluator: Evaluator | None = None) -> None:
        self.coordinator = coordinator
        self.evaluator = evaluator or Evaluator()

    def run_until_empty(
        self,
        problem: ProblemDefinition,
        snapshot: SchedulingSnapshot,
        max_decisions: int = 100,
    ) -> tuple[SchedulingSnapshot, EvaluationMetrics, list[ExecutionPlan]]:
        executor = Executor()
        plans: list[ExecutionPlan] = []
        current = snapshot
        while current.tasks and len(plans) < max_decisions:
            plan = self.coordinator.plan_one(problem, current)
            plans.append(plan)
            self.evaluator.record(plan)
            current = executor.apply(problem, current, plan)
        return current, self.evaluator.metrics, plans
```

第 8 章 如何把一种新算法接进母框架

算法	只替换/新增什么	不应重写什么
新派工规则	ObjectiveEvaluator 或 RuleDecisionEngine	状态、路径、安全、评估
MILP/CP-SAT	ExactSolverAdapter 的建模/求解/解码	候选、执行、评估接口
GA/VNS/ALNS	编码、repair、objective、neighbor/算子	问题定义和统一指标
DQN/PPO	状态编码、动作编码、策略适配、Experience/Learning	硬约束、安全盾、实验管理
MARL	联合观测/动作、通信、信用分配	底层冲突和最终安全
A*/CBS/SIPP	PathPlanner/ConflictResolver	上层车辆任务接口
数字孪生	DigitalTwinAdapter 的读状态/下发	算法接口和评估口径

接入 PPO（SB3）的最小思路

```
# 1. 保持环境输出统一 observation / action / reward
# 2. 用 ConstraintChecker 生成 action_mask（如使用支持掩码的算法）
# 3. SB3 负责内部 rollout 和网络更新
model = PPO("MlpPolicy", env, ...)
model.learn(total_timesteps=...)

# 4. 部署时仍经过母框架外围
observation = state_builder.build(...)
raw_action, _ = model.predict(observation, deterministic=True)
safe_action = safety_shield.validate_or_replace(raw_action)
executor.dispatch(safe_action)

# 关键：PPO 只替换 DecisionEngine/LearningModule，
# ProblemDefinition、ConstraintChecker、PathPlanner、SafetyShield、Evaluator 继续复用。
```

研究级实验最低清单

类别	最低要求
基线	规则 + 一个经典优化/搜索基线 + 你主张的方法
随机性	多个随机种子；报告均值、标准差和最差分位
效率	完工期、吞吐、延误、空驶、能耗、计算时间
安全	非法动作、节点/边冲突、死锁、低电量违例

类别	最低要求
泛化	不同订单密度、车数、故障率、地图和到达分布
消融	去掉奖励项、掩码、预测器、启发式或混合组件
复现	版本、配置、随机种子、训练步数、模型选择规则

第 9 章 你现在的学习与项目推进顺序

你的目标不是成为所有算法的推导专家，而是成为能判断问题、组合模块、验证结果的 AGV 调度研究者。

阶段	做什么	完成标志
A. 母框架	反复画出第 2 章链路；运行配套代码	能解释每个模块输入输出
B. 可靠基线	规则派工 + Dijkstra/A* + 预约表	多场景不报错，指标可重复
C. 原理最小集	Q-learning 看懂 TD 更新；PPO 看懂策略/价值/优势	能说清训练更新了什么
D. 单点替换	只换 DecisionEngine 为 PPO 或优化器	其他模块和指标不变
E. 研究问题	明确创新落在状态、约束、决策、路径、混合或评估哪一层	有强基线和消融方案
F. 多车进阶	CBS/预约/PIBT + MARL/HRL 按需	冲突、安全和泛化先于 reward
现在最值得内化的母句 算法模块只负责在合法候选中做更好的选择；约束模块保证能做；路径和冲突模块保证走得通；安全模块保证敢执行；评估模块证明真的更好。		

附录 A 常用缩写速查

缩写	英文	中文/作用
MILP	Mixed-Integer Linear Programming	混合整数线性规划
CP-SAT	Constraint Programming + SAT	约束规划/布尔搜索求解
MAPF	Multi-Agent Path Finding	多智能体路径规划
CBS/ECBS	Conflict-Based Search	基于冲突的最优/有界次优搜索
SIPP	Safe Interval Path Planning	安全时间区间路径规划
ALNS	Adaptive Large Neighborhood Search	自适应大邻域搜索
MDP	Markov Decision Process	马尔可夫决策过程
DQN	Deep Q-Network	深度 Q 网络
PPO	Proximal Policy Optimization	近端策略优化
SAC	Soft Actor-Critic	最大熵 Actor-Critic
MARL	Multi-Agent Reinforcement Learning	多智能体强化学习
CTDE	Centralized Training, Decentralized Execution	集中训练、分散执行
HRL	Hierarchical Reinforcement Learning	分层强化学习
MPC	Model Predictive Control	模型预测控制
DRO	Distributionally Robust Optimization	分布鲁棒优化
CVaR	Conditional Value at Risk	条件风险价值

附录 B 代码运行与扩展

```
# 运行完整母模块示例
python agv_algorithm_mother_modules_v2.py

# 运行自动测试
python -m unittest -v test_agv_algorithm_mother_modules_v2.py

# 推荐扩展顺序
```

```
# 1) 替换 build_demo() 为你的 AGV 地图、车辆和任务  
# 2) 扩展 ConstraintChecker  
# 3) 固定 Evaluator 指标  
# 4) 再替换 DecisionEngine  
# 5) 最后接数字孪生/实车 Executor
```

配套代码定位

PDF 内代码用于理解与批注；完整、可运行、可测试的版本在同目录
agv_algorithm_mother_modules_v2.py 中。